

Activity Modeling

Student Study Edition · Golden Master design · 13 structured lessons with original course visuals, examples, activities, and study aids.

01 · Behavior Modeling

02 · Object State
Modeling Foundations

03 · States, Events,
Transitions & Guards

04 · State Machine
Case Studies

05 · Composite States
& Design Steps

06 · Activity Modeling
Foundations

TOPIC 09 · STUDY MAP

Contents

Use this map to move through the topic in the same lesson order as The Educator course pages.

01	Behavior Modeling	3
02	Object State Modeling Foundations	6
03	States, Events, Transitions & Guards	8
04	State Machine Case Studies	11
05	Composite States & Design Steps	17
06	Activity Modeling Foundations	20
07	Activities, Decisions, Guards & Flow	22
08	Fork, Join, Concurrency & Object Flow	25
09	Swimlanes & Responsibility	27
10	Orders, Supply & Iteration	29
11	Activity Diagram Practice Studio	31
12	State vs Activity vs Sequence	37
13	Integrated Review & Agile Guidance	39

TOPIC 09 · LESSON 01 · 1 OF 13 LESSONS

TOPIC 09 · INTEGRATED LESSON 01

Behavior Modeling

Two complementary views of dynamic software: **state modeling** explains how one object reacts over its lifetime; **activity modeling** explains how work flows across actions, decisions, parallel paths, objects, and responsibilities.

LEARNING FOCUS

Select and construct the right UML behavioral model from requirements, then connect behavioral discoveries back to software design.

PRACTICE LAB · TOPIC 09**Mission 09 — Make the Workflow Visible**

Model CareFlow across 14 stages: choose the right behavioral view, structure actions and control flow, use decisions and guards correctly, distinguish merge from join, model concurrency and object flow, assign swimlanes, and review flawed activity diagrams.

MENTAL MODEL FIRST

Three questions, three UML views

Sequence

Conversation: Who sends what message to whom, and in what order?

State

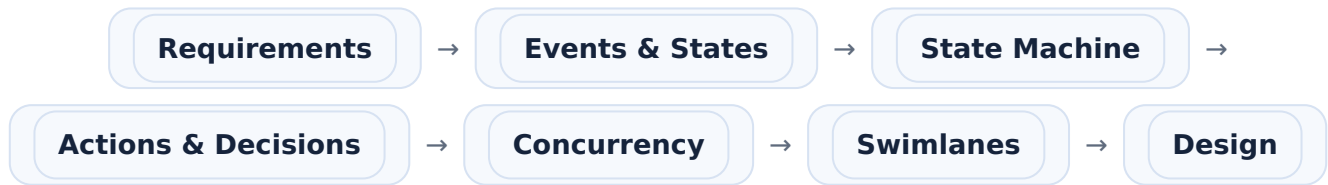
Lifetime: What condition is one object in, and what event changes it?

Activity

Workflow: What work happens next, under what condition, in parallel, and by whom?

LOGICAL LEARNING PATH

From reactive object to system workflow



This sequence prevents a common mistake: drawing notation before deciding what behavioral question the model must answer.

CORE LECTURE

Why combine these ideas?

- ✓ **Object State Modeling (OSM)** addresses event-driven, state-dependent reactions to external stimuli.
- ✓ **Activity Modeling** addresses transformational behavior: sequencing, branching, information/object flow, concurrency, synchronization, and workflow across subsystems.
- ✓ They share behavioral concepts such as transitions and guards, but their modeling focus is different.
- ✓ Together with sequence diagrams, they form a practical toolkit for describing dynamic system behavior.

REMEMBER

Do not ask “Which UML diagram do I like?” Ask “What behavior am I trying to explain?”

CONCEPT CHECK Choose before you draw

QUICK CHALLENGE

Which model?

A library book can be Available, Reserved, or Borrowed. An online order moves through payment, stock allocation, packing, and shipping. Which view fits each?

Reveal answer

Book: state machine—the same object changes condition over time. **Order processing:** activity diagram—the focus is workflow across actions and responsibilities.

TOPIC 09 · LESSON 02 · 2 OF 13 LESSONS

LESSON 02 · REACTIVE BEHAVIOR

Object State Modeling Foundations

Model the state-dependent reactions of an object to events or external stimuli.

LEARNING FOCUS

Identify events, states, and state-dependent behavior from a requirement story.

CORE LECTURE

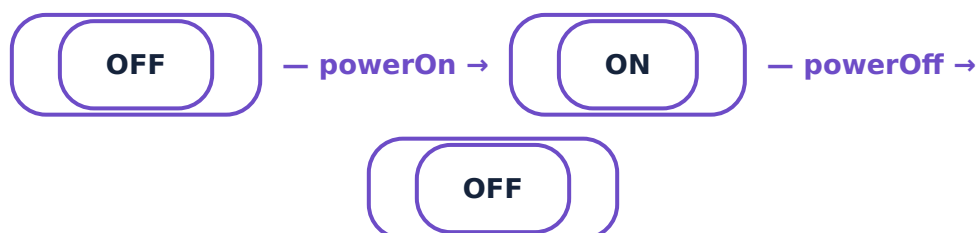
What is object state modeling?

It is the identification, modeling, analysis, design, and specification of **state-dependent reactions** of objects to external stimuli.

- ✓ What external stimuli or events matter?
- ✓ What states can the object occupy?
- ✓ How can we tell which state the object is in?
- ✓ How does the reaction to an event depend on the current state?
- ✓ How should complex states be represented and checked?

SIMPLE INTUITION

The same event can mean different things



The behavior is not just “what event occurred?” It is “what event occurred while the object was in which state?”

BASIC DEFINITIONS

Event and state

- ✓ **Event:** a happening of interest or a request to a subsystem, object, or component.
- ✓ **State:** a named abstraction of an object's condition or situation, entered or exited because of an event.

DIAGNOSTIC RULE

If an object responds differently to the same request depending on its current condition, state modeling is probably valuable.

THINK

Car example

An engine may be On/Off; a transmission may be Park/Reverse/Neutral/Forward; a brake may be Released/Applied. These are separate stateful concerns. Decide which object owns each state before drawing one giant diagram.

TOPIC 09 · LESSON 03 · 3 OF 13 LESSONS

LESSON 03 · READ THE LANGUAGE

States, Events, Transitions & Guards

The minimum vocabulary needed to read and construct a UML state machine correctly.

LEARNING FOCUS

Distinguish state, event, transition, guard, start, and final state—and label them meaningfully.

CORE NOTATION

States

- ✓ At a given point in time, the modeled object is in a state and remains there until a transition occurs.
- ✓ A state is shown as a rounded rectangle.
- ✓ Useful state names often describe an ongoing condition: Waiting, Dialing, Powered.
- ✓ A filled black circle is the initial node; a bullseye-style node is the final node.

VISUAL REFERENCE

UML State Diagram Symbols

Use this summary as a visual reference while reading and constructing state machines. Focus on what each symbol means before memorizing its shape.

Symbols

The following table contains the symbols of the state diagrams.



State diagrams show how an object changes state over time in response to events.

Simple Symbols
Clear Meaning
Better Models!

Search...

Name / Symbol	Description & Usage	Example
State Represents a state of an object		- The state of an object is symbolised by a rectangle with rounded corners. - The state is named within this symbol.
Initial (Starting point) Represents the initial point of the state diagram		- The initial point of the state diagram is shown with a filled circle. - It is identical to the object creation. - Only one starting point per state diagram is permitted and must be present. - The arrangement of the starting point is optional.
Final (End point) Represents the end point of the state diagram		- The chain of state transitions ends with object destruction. - The end point is represented by a filled circle surrounded by a concentric circle. - For endlessly running processes, this symbol may be missing in the drawing, but it may also be entered multiple times. - The token may return to the end of the activity in the parent diagram that called the sub-diagram.
Transition Represents a state transition		- The transition is marked with an arrow. - The arrow is labelled with the name of the trigger that changes the object state. - You can specify a restriction [Guard] in square brackets, which means that the object status is only changed if it is fulfilled. - In addition, an activity list to be executed during the transition can be specified after '/'. - Guard and activity list are optional, the trigger can also be missing. - at the transition from initial and if an ANY trigger is modelled.



UML State Diagram Symbols

Model behavior. Understand change. Design better systems.

From States to Better Software

State notation at a glance: state, initial node, final node, and transition. Transition labels may combine an event, a guard condition, and an action.

CORE NOTATION

Transitions and guards

A **transition** is an instantaneous change of state in response to an event. A **guard** is a Boolean condition that must be true for that transition to occur.



Read it as: while the Book is Available, event reserve moves it to Reserved only if memberValid is true.

TIME EVENTS

Behavior can be time-triggered

Transitions may be triggered by elapsed time, such as after(30s). Traffic lights are a natural example: Green → Yellow → Red as timers expire, while sensor events and guards can create alternative behavior.

COMMON MISTAKE

A guard is not a state. “Payment approved” may be a condition guarding a transition; “Paid” may be a state if the object remains in that condition over time.

MINI CHECK**Event or state?**

submitted usually describes an event; **Under Review** describes a persistent state. Ask whether it happens or whether the object can remain there.

TOPIC 09 · LESSON 04 · 4 OF 13 LESSONS

LESSON 04 · READ LIFECYCLES PROGRESSIVELY

State Machine Case Studies

Three domains, three levels of state-modeling complexity. Read each diagram as the lifecycle of one subject—not as a workflow of people and departments.

LEARNING FOCUS

Progress from a mostly linear lifecycle, to guarded approval behavior, to a richer patient encounter with nested and concurrent state detail.

1 · Food Order

Start with states, events and exception exits.

2 · Registration Request

Add guards, eligibility and conditional approval.

3 · Patient Encounter

Finish with composite/concurrent state behavior.

READING RULE

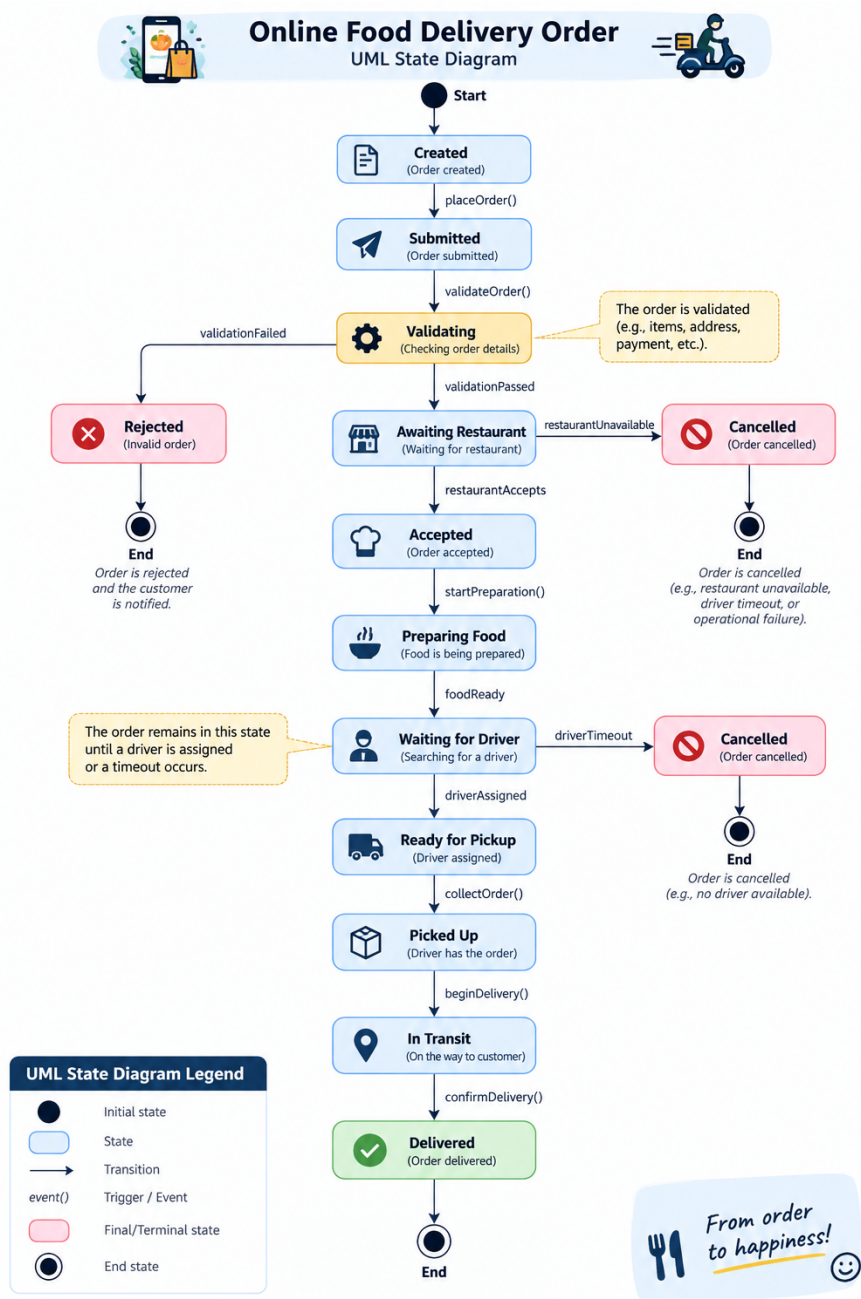
At every node ask: What is the subject now? On every arrow ask: What event or condition makes that subject change state?

BASIC LIFECYCLE

Online Food Delivery Order

The subject is the **Order**. Follow its condition from creation through validation, restaurant acceptance, food preparation, driver assignment, pickup, transit and delivery. Rejected and Cancelled are alternative terminal outcomes.

States**Event-labeled transitions****Alternative terminal states****Waiting state**



Read first: trace only the central successful path. **Read second:** inspect where validation failure, restaurant unavailability or driver timeout diverts the order.

What to learn here: a state diagram is about the Order's condition. Food preparation and driver search may be activities elsewhere, but here they matter only when they change the Order's state.

INTERMEDIATE LIFECYCLE

University Course Registration Request

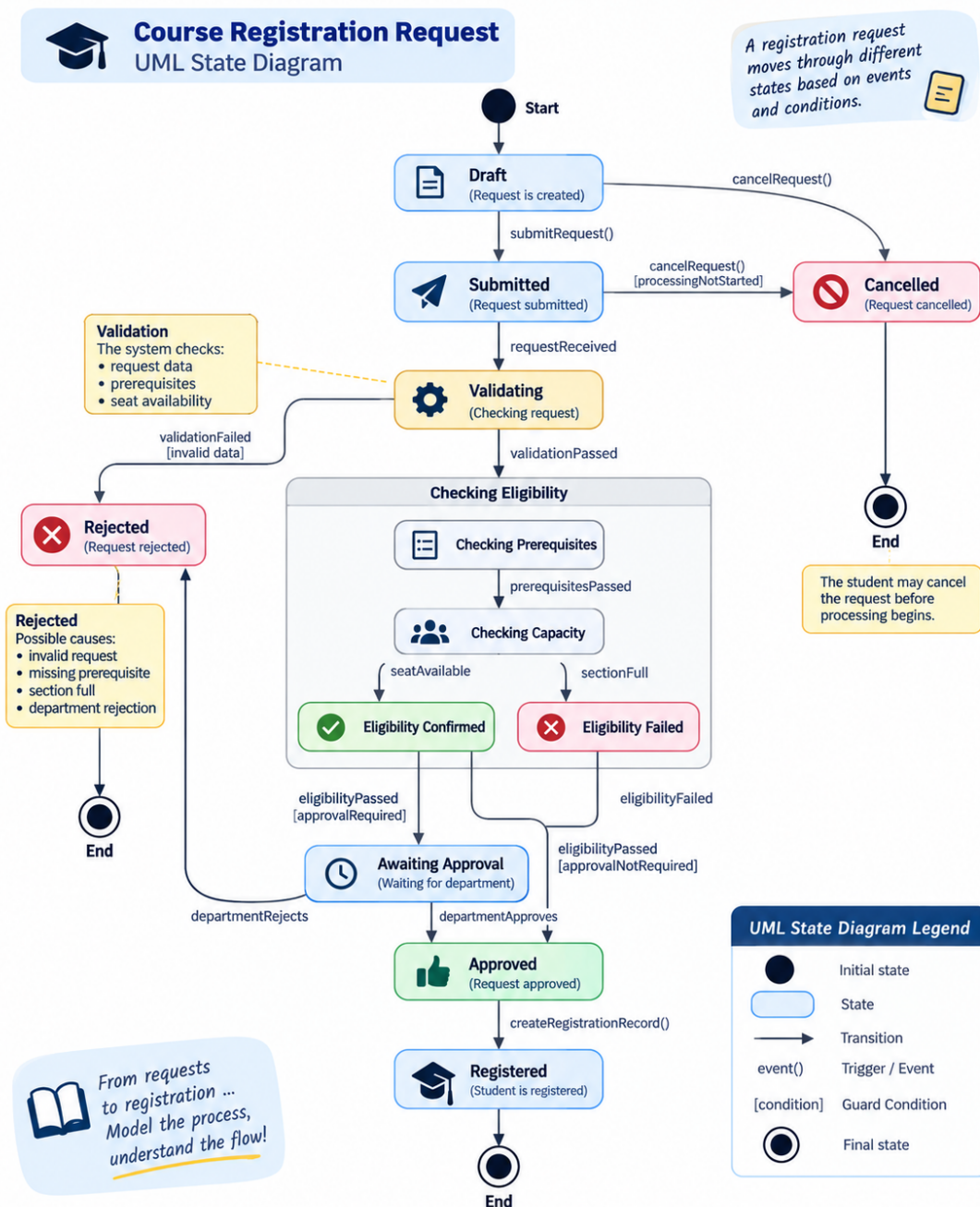
The subject is the **Registration Request**. This case adds validation, eligibility, guards, optional departmental approval, cancellation, rejection and successful registration.

Guards

Conditional approval

Composite eligibility

Multiple final outcomes



Read first: Draft → Submitted → Validating. **Then:** study the guarded eligibility and approval branches. Treat prerequisite/capacity checking as state detail, not as a replacement for an Activity Diagram.

What to learn here: events trigger transitions; guards decide whether a triggered transition is allowed. “Approval required” is a condition, while “Awaiting Approval” is a persistent state.

ADVANCED LIFECYCLE

Hospital Patient Encounter

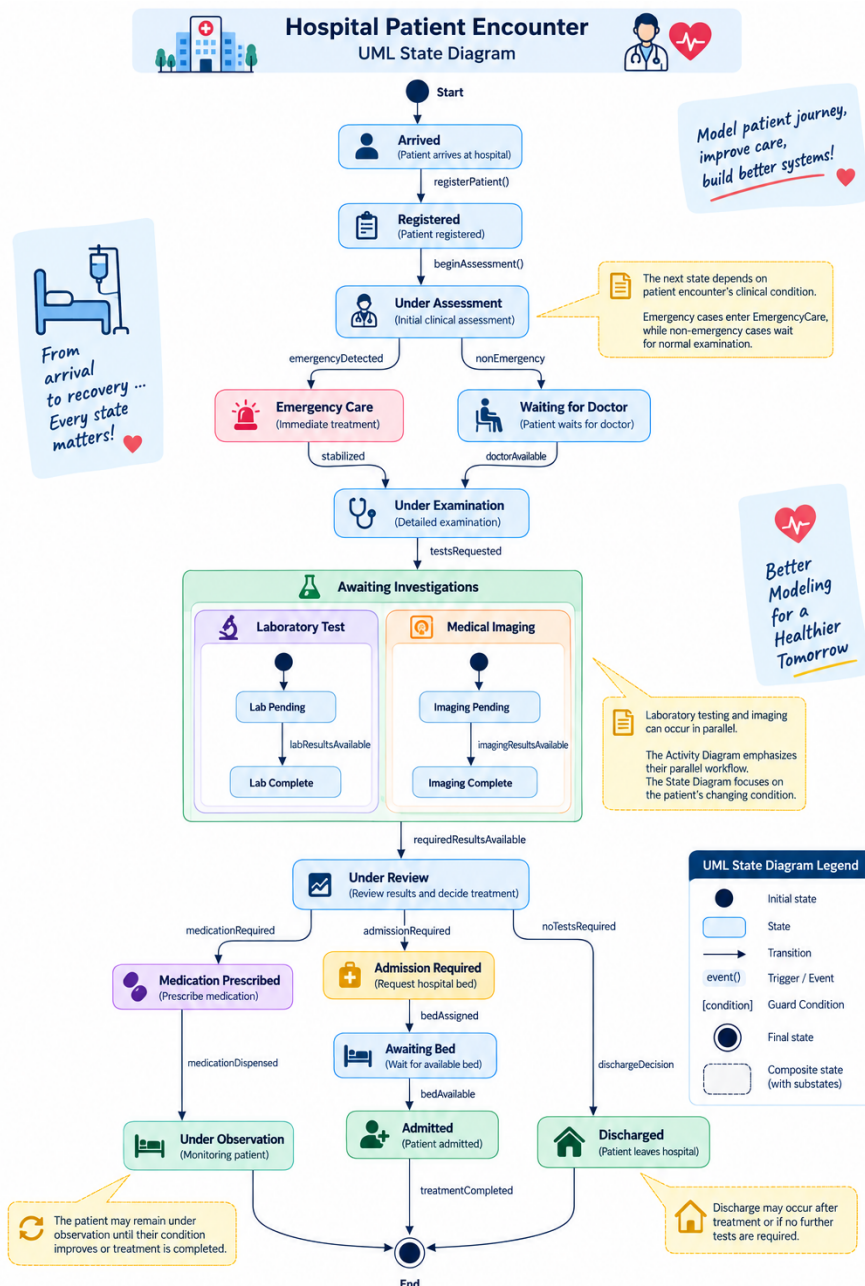
The subject is the **Patient Encounter**. Emergency vs normal care changes the route, investigations can contain parallel substates, and treatment decisions lead to observation, admission or discharge.

Composite state

Concurrent substates

Clinical branching

Converging outcomes



Read first: Arrived → Registered → Under Assessment → Under Examination. **Then:** inspect the composite investigations region and the alternative treatment/admission/discharge outcomes.

What to learn here: use composite or concurrent state detail only when it explains the subject's condition. Do not turn a state machine into an organizational workflow.

PROGRESSION CHECK

What became more sophisticated?

Food Order
basic lifecycle + exceptions

Registration
guards + conditional approval

Hospital
composite/concurrent state detail

Do not memorize these domains. Learn the progression: identify a subject, name persistent states, label state-changing events, add guards only when needed, and introduce nested/concurrent detail only when it improves understanding.

TOPIC 09 · LESSON 05 · 5 OF 13 LESSONS

LESSON 05 · FROM REQUIREMENTS TO DESIGN

Composite States & Design Steps

Scale state machines without losing clarity, then feed discovered behavior back into the design class.

LEARNING FOCUS

Classify behavioral information, use nested substates when needed, and derive design operations from transitions.

SYSTEMATIC EXTRACTION

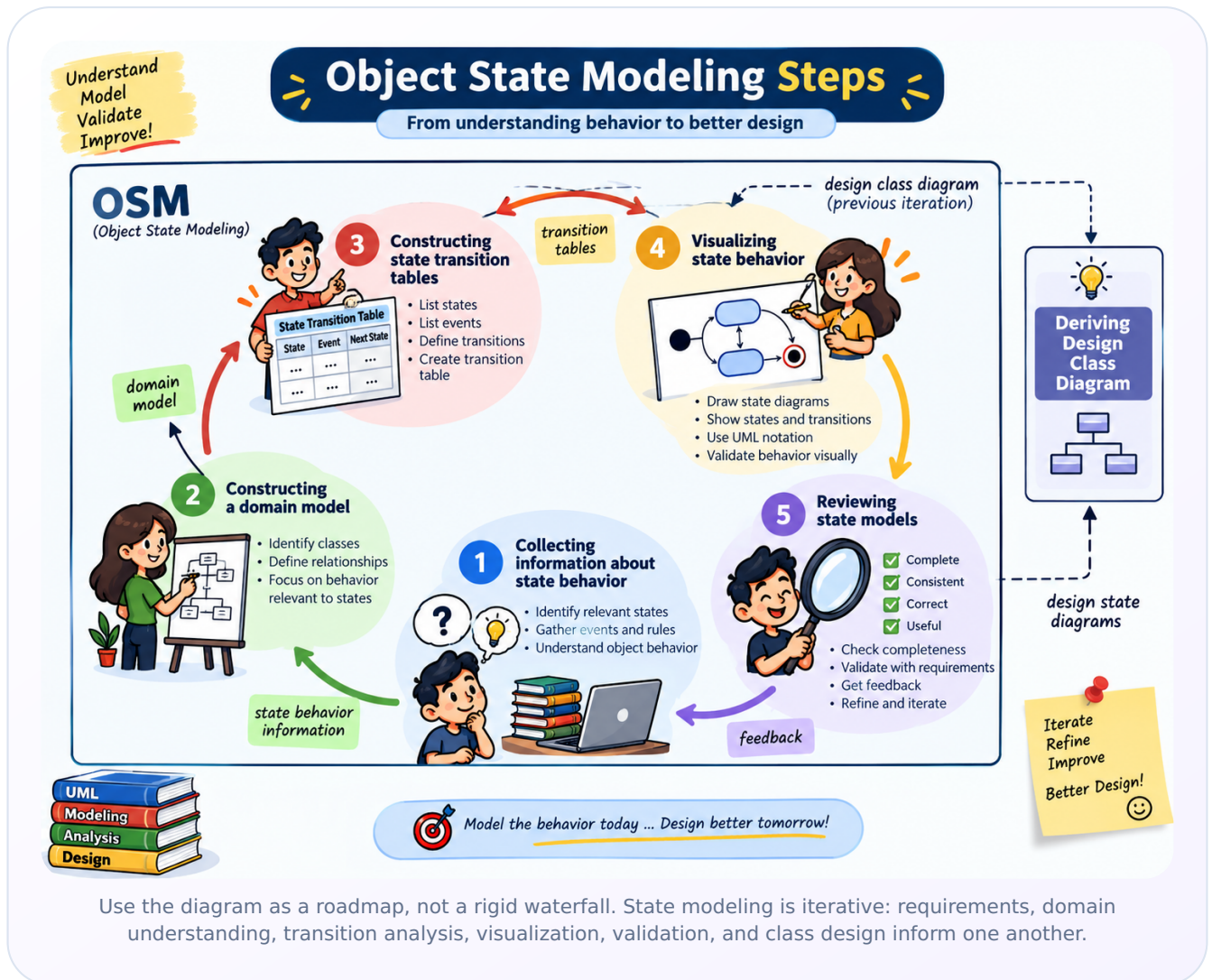
What to look for in requirements

- ✓ **Event:** something of interest happened — e.g., an application is submitted.
- ✓ **State:** a mode of operation — e.g., cruise control is Activated/Deactivated.
- ✓ **Guard:** a condition governing event processing — e.g., turn on AC if temperature is high.
- ✓ **Response/action:** work associated with the event — e.g., set the desired cruising speed.

VISUAL ROADMAP

Object State Modeling Steps

This visual connects the state-modeling workflow to the wider design process: collect behavior information, build the domain understanding, construct transition tables, visualize the state behavior, review it, and feed the results back into class design.



PRACTICAL WORKFLOW

Object state modeling steps

- 1 Select the subject object with meaningful state-dependent behavior.
- 2 Collect events, states, guards, and responses from requirements.
- 3 Connect states with event-triggered transitions.
- 4 Add guards and actions only where they clarify behavior.
- 5 Use composite/nested states when one state contains meaningful sequential substates.

6 Review reachability, missing transitions, and contradictory behavior.

7 Update the design class with operations implied by important transitions.

DESIGN CONNECTION

State model → class operations

For a **CruiseControl** subject, transition labels may reveal operations such as `onOffButtonPressed()`, `leverDown()`, `leverUp()`, `brakeApplied()`, and `setDesiredSpeed()`.

COMPOSITE STATE

Hide detail until it matters

A state can contain nested substates. For example, a transmission's **Drive** state may contain **First**, **Second**, and **Third**. This keeps the top-level machine readable while preserving detailed behavior.

AGILE PRINCIPLE

Work closely with users, keep behavior models high-level and visual, and do only enough modeling to reduce uncertainty and guide working software.

TOPIC 09 · LESSON 06 · 6 OF 13 LESSONS

LESSON 06 · SHIFT THE LENS

Activity Modeling Foundations

Now move from the lifetime of one object to the flow of work through a process or system.

LEARNING FOCUS

Understand what activity modeling adds beyond a state machine and when a workflow view is the better abstraction.

THE BRIDGE

Same behavioral family, different focus

State machine

Transitions are commonly driven by events affecting one stateful subject.

→ change the question →

Activity diagram

Flow commonly advances when work/ actions complete, with decisions and concurrent paths.

CORE LECTURE

What activity modeling covers

- ✓ Complex information-processing activities and operations.
- ✓ Information flow and object flow among activities.
- ✓ Conditional branching according to decisions.
- ✓ Forking, concurrency, joining, and synchronization.

- ✓ Workflow among departments, classes, or subsystems.

WHY USE IT?

Understand current work, then design better work

Activity modeling can describe processing in the existing organization to understand the business, then describe processing in the proposed solution during system design.

USE-CASE CONNECTION

Useful scope

Activity diagrams can describe a use case, a complicated method, or workflow spanning several use cases. Their particular strength is making parallel work visible.

BOUNDARY

Do not use an activity diagram to explain how one object behaves over its lifetime. That is exactly where the state machine you just learned is stronger.

TOPIC 09 · LESSON 07 · 7 OF 13 LESSONS

LESSON 07 · WORKFLOW GRAMMAR

Activities, Decisions, Guards & Flow

Read the notation used to express work, sequencing, alternatives, iteration, and completion.

CORE LECTURE

Notation essentials

- ✓ **Action/activity:** a task that needs to be done.
- ✓ **Control flow:** connects actions in execution order.
- ✓ **Decision:** a diamond that selects among guarded alternatives.
- ✓ **Merge:** reunites alternative routes.
- ✓ **Initial / activity-final / flow-final nodes:** define entry and termination.
- ✓ **Object flow:** shows information or objects moving among actions.

VISUAL REFERENCE

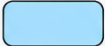
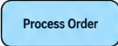
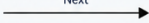
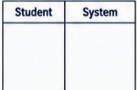
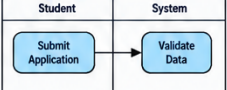
UML Activity Diagram Notation Summary

Use this large reference as a decoding sheet while reading and drawing activity diagrams. Focus first on the core symbols, then compare the example column to see each notation in context.

UML Activity Diagram Notation Summary

Activity diagrams model the flow of activities, decisions and parallel execution.

Used to represent the workflow of a process.

Notation	Description	UML Symbol	Example (Labeled)
 Activity	Represents a set of actions or an activity.		
 Action	A single task to be performed (atomic step).		
 Control Flow	Shows the sequence of execution.		
 Object Flow	Shows the flow of an object (from one activity to another).		
 Initial Node	Represents the beginning of a set of actions.		
 Activity Final Node	Stops all control flows and object flows in an activity.		
 Object Node	Represents an object that is connected to a set of object flows.		
 Decision Node	Represents a test condition to ensure that the control flow or object flow only goes down one path.		
 Merge Node	Brings back together different decision paths that were created using a decision node.		
 Fork Node	Splits behavior into a set of parallel or concurrent flows.		
 Join Node	Brings back together a set of parallel or concurrent flows.		
 Swimlane (Partition)	A way to group activities performed by the same actor or to group activities in a single thread.		



Tip: Use a consistent notation and clear labels to make your activity diagrams easy to read and understand.



Model the process, not the implementation details.

Activity notation reference. Keep this visible while practicing decision/merge, fork/join, object flow, and swimlane diagrams.

CRITICAL DISTINCTION

Decision is not concurrency

Decision → Merge
Choose one alternative according to guards.

Fork → Join
Start parallel work and synchronize it later.

EXAMPLE

Beverage preparation

Find beverage → if coffee is found, prepare filter/water and brew; otherwise obtain cola; obtain cups; then drink. The example shows how guarded alternatives and sequencing turn a verbal procedure into a visible workflow.

REMEMBER

Guards belong on alternative outgoing flows. Make conditions mutually understandable and avoid unlabeled diamonds when the choice is not obvious.

TOPIC 09 · LESSON 08 · 8 OF 13 LESSONS

LESSON 08 · PARALLEL BEHAVIOR

Fork, Join, Concurrency & Object Flow

Model work that can proceed in parallel, the point where it must synchronize, and the information moving through the process.

CORE LECTURE

Parallel work

- ✓ A **fork** splits one flow into concurrent threads.
- ✓ A **join** synchronizes concurrent threads before continuing.
- ✓ Fork/join means parallel obligations; decision/merge means alternatives.
- ✓ An object node such as :NewOrder or :Invoice can show object flow between actions.

VISUAL MEMORY HOOK

Split work, then wait

1  **FORK**
start
together

Prepare items

Authorize payment

2  **JOIN**
continue
when
synchronized

DESIGN JUDGMENT

Do not invent concurrency

Use a fork only when activities are logically independent enough to proceed concurrently. Drawing parallel branches merely to make a diagram look sophisticated creates false system requirements.

EXAM CUE

Decision = one path. Fork = multiple paths. Merge = alternatives reunite. Join = parallel work synchronizes.

TOPIC 09 · LESSON 09 · 9 OF 13 LESSONS

LESSON 09 · RESPONSIBILITY

Swimlanes & Responsibility

Make handoffs visible by partitioning workflow according to the actor, class, department, or subsystem responsible.

CORE LECTURE

Responsibility partitions

- ✓ Activity diagrams often involve several classes or organizational roles.
- ✓ Swimlanes divide the diagram into responsibility zones.
- ✓ Each zone represents a particular class, department, actor, or subsystem.
- ✓ Cross-lane flows expose handoffs—often the points where delay, coupling, or ambiguity occurs.

LECTURE-INSPIRED EXAMPLE

Course registration

Student
Submit registration request

CourseSection
Check prerequisites
Verify capacity

Registrar/System
Check permission
Complete registration

Guarded “not ok” paths explain rejection; successful checks converge before registration completes.

INTERPRETATION SKILL

Read the handoffs

Do not only trace arrows. Ask: Who owns this action? Where does responsibility move? Is information/object flow clear at the boundary?

REMEMBER

Swimlanes add accountability to workflow. They answer “who does it?” while the activity flow answers “what happens next?”

TOPIC 09 · LESSON 10 · 10 OF 13 LESSONS

LESSON 10 · PUT NOTATION TO WORK

Orders, Supply & Iteration

Use real business rules to combine decisions, repeated processing, synchronization, and responsibility.

RECEIVING AN ORDER

Line items + payment

- ✓ Receive the order and process each line item.
- ✓ If an item is in stock, assign it; if it needs reorder, trigger Reorder Item.
- ✓ Authorize payment on a guarded path; failure may cancel the order.
- ✓ Dispatch only when stock is assigned to all line items and payment is authorized.

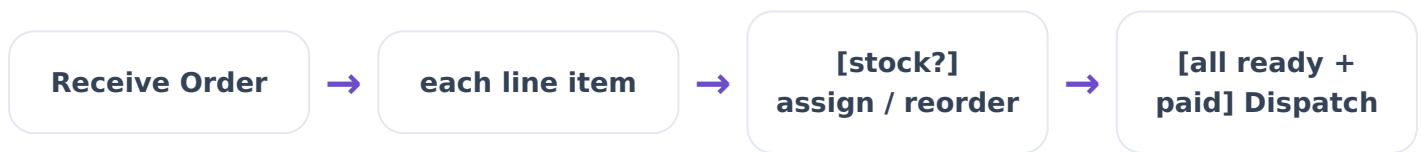
RECEIVING SUPPLY

Incoming goods + outstanding orders

- ✓ Choose outstanding order items that incoming supply can fill.
- ✓ Assign goods to orders and repeat for each selected item.
- ✓ If all outstanding items for an order are filled, the order may be dispatched.
- ✓ Put remaining goods into stock.

PATTERN

Iteration + guards + synchronization



MODEL THE RULE, NOT DECORATION

Every guard, loop, fork, and join should correspond to a real requirement. If you cannot explain its business meaning, reconsider it.

TOPIC 09 · LESSON 11 · 11 OF 13 LESSONS

LESSON 11 · LEARNING BY DOING

Activity Diagram Practice Studio

You have already seen state lifecycles. Now deliberately change the question: model **work, decisions, concurrency and responsibility** rather than the condition of one object.

LEARNING DESTINATION

Translate realistic process descriptions into UML Activity Diagrams using actions, decisions, guards, fork/join, final outcomes and swimlanes—without duplicating the state-machine view.

Do not redraw the State Diagram. A state model asks “What condition is the subject in?” An activity model asks “What work happens next, who performs it, which path is chosen, and what can happen in parallel?”

1 · University Registration

Start with clean parallel checks + conditional approval.

2 · Food Delivery

Add operational concurrency + cancellation.

3 · Hospital Treatment

Finish with multi-role clinical branching.

PRACTICE RULE

Read the story and draw before revealing the image. Your layout may differ from the suggested solution, but the workflow semantics must be correct.

ACTIVITY A · FOUNDATION

University Course Registration

A student submits a request to register for a course section. The system verifies prerequisites and available seats; these checks can run independently. If either fails, the request is rejected. If both succeed, the system checks whether departmental approval is required. Approval leads to registration; no approval requirement leads directly to registration; rejection leads to a student notification.

Decision + guards

Fork / Join

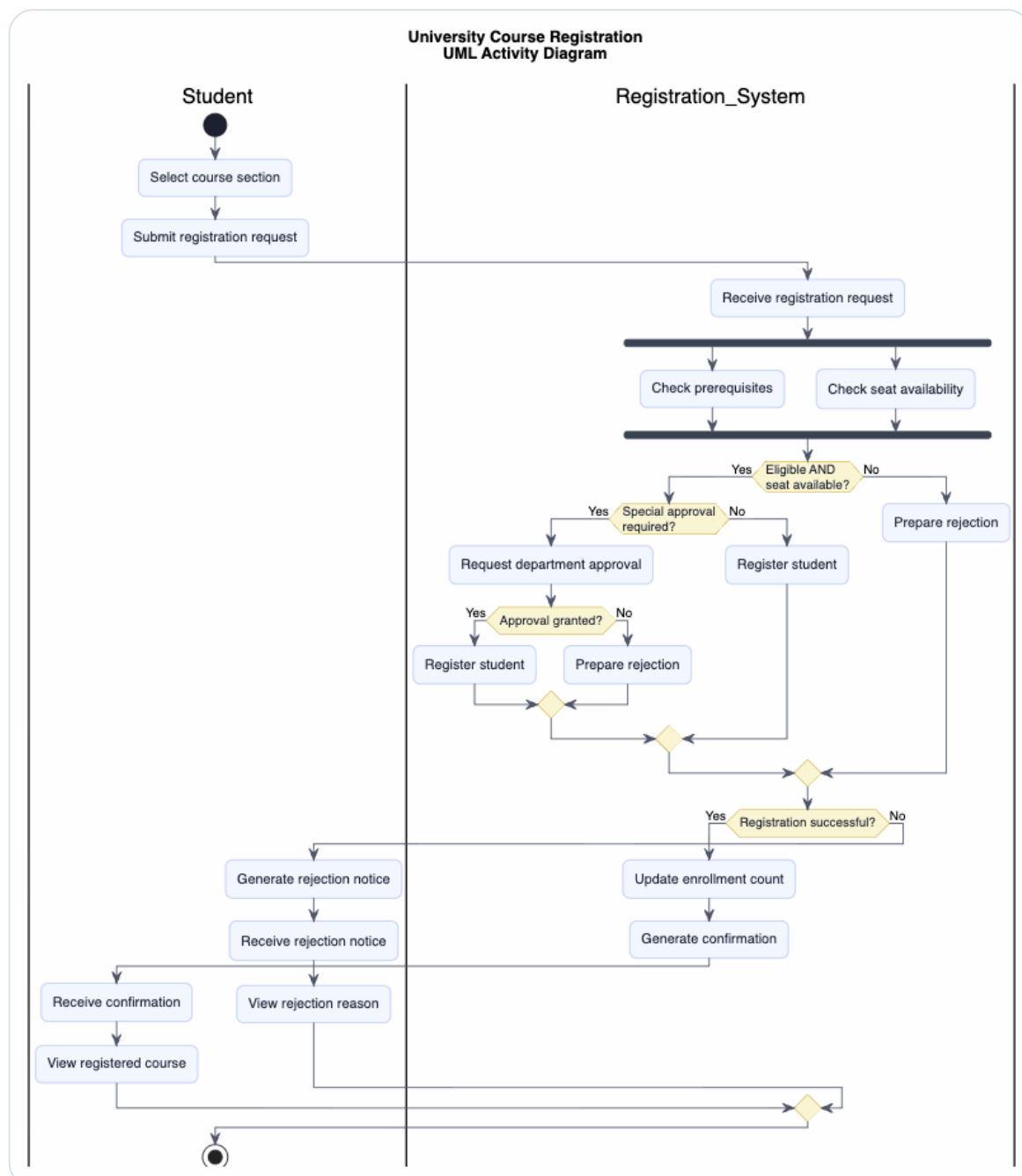
Student

Registration System

Department

View in this order: (1) locate the Student/System handoff, (2) find the fork and join for prerequisite/capacity checks, (3) trace the approval decision, then (4) compare success and rejection outcomes.

Reveal Activity Diagram · University Registration



Activity lens: responsibilities and parallel validation. Do not interpret actions such as “Check prerequisites” as persistent states of the registration request.

02

ACTIVITY B · INTERMEDIATE

Online Food Delivery Order

A customer places an order. After validation and restaurant availability checks, restaurant preparation and driver assignment can proceed concurrently. Pickup requires the needed parallel work to complete. If a driver cannot be assigned or another required condition fails, the process follows a cancellation/refund path. Successful delivery ends with customer confirmation or rating.

Parallel operations

Cancellation branch

Customer

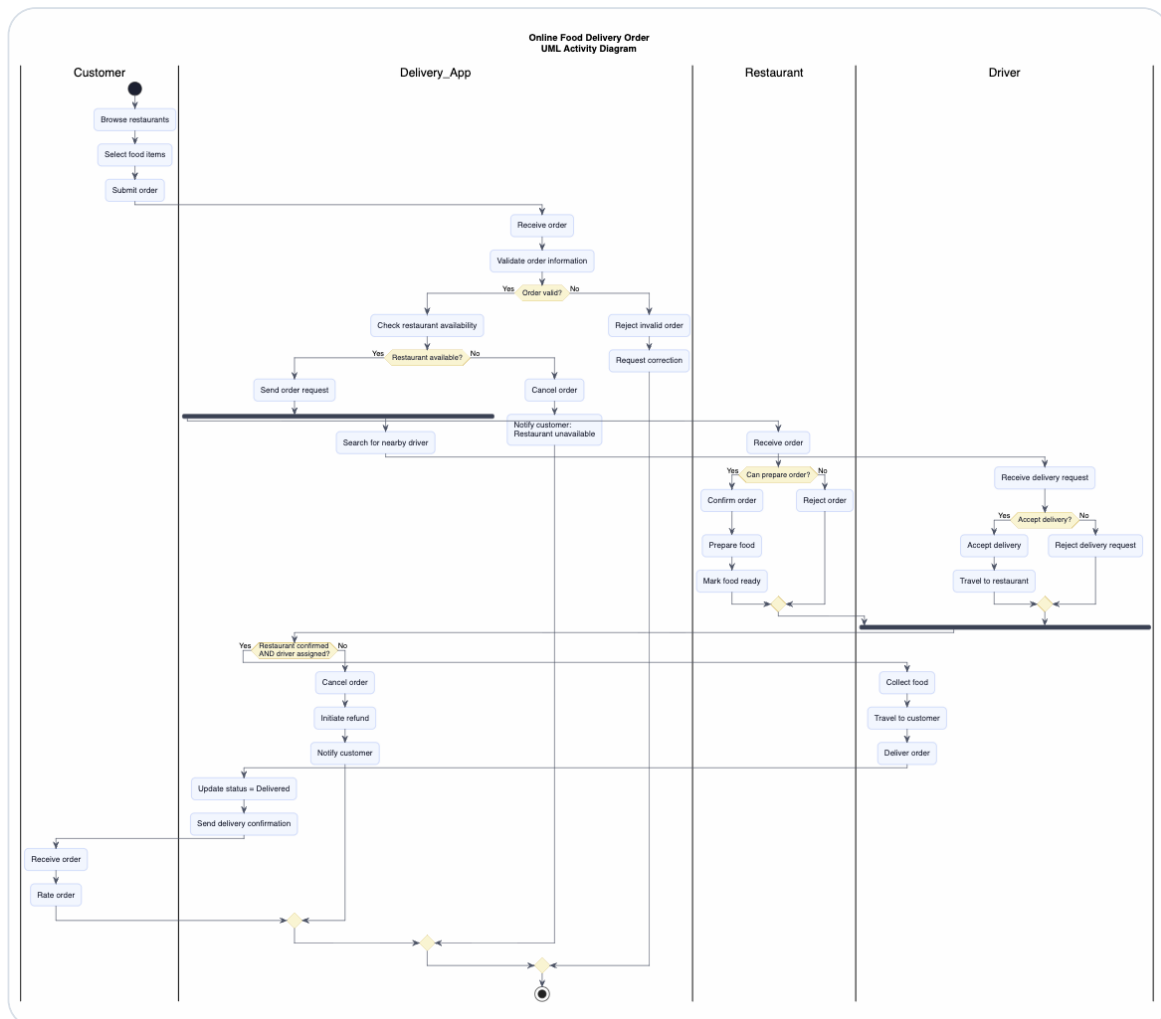
Delivery App

Restaurant

Driver

View in this order: (1) follow validation, (2) identify the parallel restaurant/driver work, (3) locate the synchronization point before pickup, and (4) inspect the cancellation path separately.

Reveal Activity Diagram · Food Delivery



Activity lens: the diagram explains coordinated work. Earlier, the State Diagram explained the Order object's lifecycle; this diagram instead shows who does what and what can happen concurrently.

03

ACTIVITY C · ADVANCED

Hospital Patient Admission and Treatment

A patient arrives, registers and is assessed. Emergency and non-emergency cases follow different routes before examination. Required laboratory tests and imaging may run in parallel. After results are available, the clinical team decides whether the patient is discharged, treated/observed or admitted, with administrative bed processing when admission is required.

Emergency decision

Parallel investigations

Patient

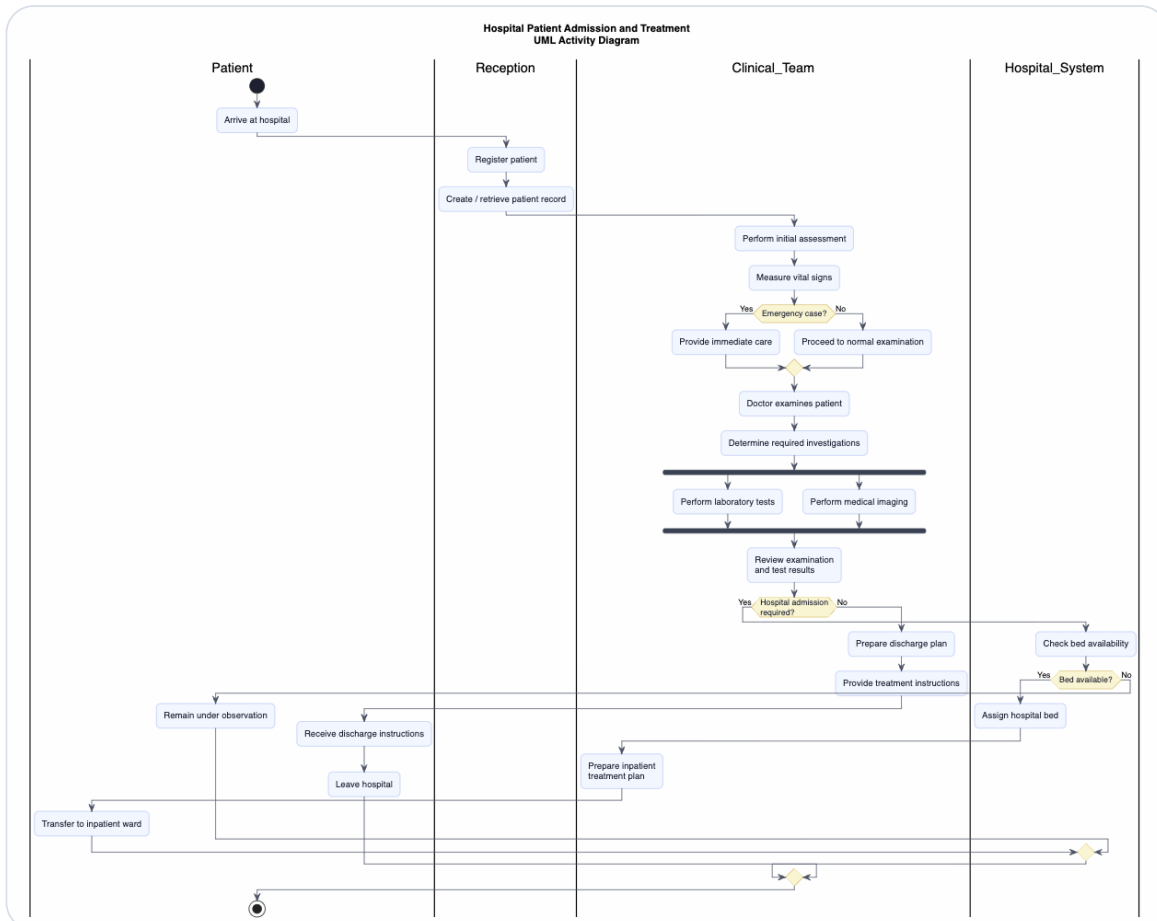
Reception

Clinical Team

Administration

View in this order: (1) trace arrival and triage, (2) separate emergency from normal care, (3) inspect parallel investigations, (4) follow the final treatment/admission decision, then (5) check responsibility handoffs.

Reveal Activity Diagram · Hospital Treatment



Activity lens: focus on clinical/administrative workflow. The earlier Patient Encounter State Diagram focused on the encounter's changing condition, including composite investigation detail.

REFLECTION

Three activity diagrams, three new workflow ideas

Registration
parallel validation + optional approval

Food Delivery
parallel operational work + cancellation

Hospital
multi-role branching + parallel investigations

The domains may be familiar from state modeling, but the information shown is intentionally different. Reusing a domain helps you see the distinction between **lifecycle** and **workflow** without duplicating the model.

TOPIC 09 · LESSON 12 · 12 OF 13 LESSONS

LESSON 12 · INTEGRATION

State vs Activity vs Sequence

The skill is not drawing more UML. The skill is choosing the smallest model that answers the behavioral question.

DECISION MATRIX

Choose by modeling question

Need to explain...	Best view	Look for...
Messages among collaborating objects	Sequence	lifelines, messages, ordering
Lifetime of one object/actor	State	states, events, transitions, guards
Business/system workflow	Activity	actions, decisions, fork/join, object flow, swimlanes

ONE SCENARIO, THREE LENSES

Online order

✓ **Sequence:** Customer → OrderController → PaymentService → Inventory.

✓ **State:** Order moves New → Paid → Packed → Shipped → Delivered/Cancelled.

✓ **Activity:** verify payment and stock, perform parallel preparation, pack, ship, and close the order.

These models are complementary. They should not be forced to duplicate each other.

MODELING CHALLENGE

Diagnose the requirement

“A support ticket may be Open, Assigned, Waiting for Customer, Resolved, or Closed.” → **State**.
“After submission, validate data, run fraud screening and credit checks in parallel, then decide.” → **Activity**.
“Show the calls among UI, controller, repository, and notification service.” → **Sequence**.

STRONG ANSWER PATTERN

Name the model, then justify it using the behavioral question—not merely the notation it contains.

TOPIC 09 · LESSON 13 · 13 OF 13 LESSONS

LESSON 13 · CONSOLIDATE

Integrated Review & Agile Guidance

Finish with model selection, construction checks, and the lightweight modeling discipline expected in software engineering practice.

STATE MODELING CHECKLIST

Can you explain one object's lifetime?

- ✓ Identify the subject object, states, events, transitions, and guards.
- ✓ Distinguish persistent states from events and Boolean conditions.
- ✓ Use nested states only when complexity justifies them.
- ✓ Trace important transitions back to design operations.

ACTIVITY MODELING CHECKLIST

Can you explain the workflow?

- ✓ Identify actions, control flow, decisions/guards, and iteration.
- ✓ Distinguish decision/merge from fork/join.
- ✓ Use object flow when information or domain objects matter.
- ✓ Use swimlanes to make responsibility and handoffs explicit.

FINAL COMPARISON

When to use what?

Sequence
object collaboration and message order

State
lifetime and event-driven reactive behavior

Activity
business/system workflow and concurrency

AGILE GUIDANCE

Do barely enough modeling

- ✓ Work closely with customers and users.
- ✓ Keep models high-level, lightweight, and visual.
- ✓ Use collaborative modeling to resolve uncertainty.
- ✓ Value working software over comprehensive documentation.

SELF-TEST

Five questions before you finish

- ✓ What behavioral question does my diagram answer?
- ✓ Does every state/action have a clear domain meaning?
- ✓ Are guards precise and attached to the right alternatives?
- ✓ Did I confuse alternatives with concurrency?

- ✓ Can a teammate understand the model without my narration?

TOPIC 09 TAKEAWAY

Behavior modeling is not three disconnected UML diagrams. It is three complementary lenses: interaction, lifetime, and workflow.